

Design Decisions and Rationale

1. Separation of Concerns (MVC-like split)

The project is split into three classes: Calculator, CalculatorUI, and Main. Calculator contains only pure arithmetic logic, with no knowledge of the UI. This makes the logic independently testable — you can call `calculator.evaluate()` without any GUI. CalculatorUI owns all Swing components and state. The reason for this split is that mixing arithmetic and display logic in one class would make both harder to modify and test.

2. State Machine for Input Handling

Three boolean flags — `waitingForSecond`, `justCalculated`, and `hasError` — form an implicit state machine that governs how each button press is interpreted. For example, pressing a digit after `=` starts a fresh number instead of appending, because `justCalculated` is true. This approach was chosen over parsing the full display string on every keypress, which would be fragile and harder to extend.

3. ButtonType Enum for Routing

Rather than comparing button labels with string literals throughout the event handler, each button is tagged with a `ButtonType` enum value at creation time. The `handle()` method switches on type, not text. This avoids bugs where a label change would silently break behavior and keeps each handler (`onDigit`, `onOperator`, etc.) focused on a single responsibility.

4. Error Locking

When an error occurs (e.g., division by zero or incomplete expression), `hasError` is set to true and all further button presses are ignored until `C` is pressed. This prevents the user from entering an undefined state by continuing to type after an error, which was a real edge case during development.

5. Number Formatting (`fmt` method)

Results are displayed as integers when the value is whole (e.g., 4.0 shown as 4) and as decimals otherwise. This was chosen because showing 4.0 for `2 + 2` looks unnatural and reduces readability. The threshold `1e15` avoids switching to scientific notation for large numbers where integer display is still readable.

6. Java Swing over JavaFX

Swing was chosen because it requires no additional build configuration (it ships with the JDK) and integrates directly with Maven without extra dependencies, keeping the project self-contained.

Main functionalities include addition, subtraction, multiplication, division, modulo, decimal input, negation, delete, clear, chained operations, and error handling such as division by zero.

The implementation uses Java Swing for the interface. `Calculator.java` performs arithmetic operations, while `CalculatorUI.java` creates the display, buttons, and event handlers.

`Main.java` launches the application using `SwingUtilities.invokeLater()` for thread-safe GUI execution.